



Git & GitHub 實務整理



GitHub

👉 你的 Repo:

<https://github.com/zekisu00?tab=repositories>

這份筆記的核心邏輯非常清晰！為了讓它更符合開發實務，我幫你微調了「Git 核心五步」的標題與描述，並將原本分散的內容整合進這五大步驟中，讓它看起來更像一份標準的「開發者手冊」。



Git 核心開發工作流(五部曲)

這是在開發專案(如 PHP、Unity 或 Web)時，最標準的操作循環：

1. `git init` —— 專案初始化

- 動作：在專案資料夾內建立 `.git` 隱藏資料夾。
- 時機：每個新專案只需要執行一次。
- 注意：執行前請確保你在正確的資料夾(如 `D:\0421-class`)。

2. `git status` —— 狀態檢查

- 動作：查看目前哪些檔案被修改、哪些還沒被追蹤(紅字)。
- 時機：隨時執行。這是最常用的指令，幫助你確認目前的工作進度。

3. `git add .` —— 加入暫存區

- 動作：將所有修改過的檔案「勾選」起來，準備打包。
- 技巧：`* git add .`：加入當前資料夾所有變動。
 - `git add 檔案名稱`：精確加入特定檔案(專業推薦)。

4. `git commit -m "訊息"` —— 建立版本

- 動作：將暫存區的檔案正式存檔，並寫下這次改了什麼。
- 必備前提：必須先完成 身分設定 (Identify) 否則會報錯：
 - `git config --global user.email "your@email.com"`
 - `git config --global user.name "yourname"`

5. git push —— 推送到遠端

- 動作：將本地的版本紀錄同步到 GitHub。
 - 完整指令：`git push 遠端名稱 分支名稱`(例如：`git push basic main`)。
-

進階補充：身分管理與紀錄

身份檢索

如果你不確定現在電腦是用誰的名義在提交，可以使用：

- `git config --list`: 查看所有設定 (包含 Email 和 Name)。
- `git config --global --edit`: 直接開啟設定檔手動修正錯誤。

歷史足跡

- `git log --oneline`: 用簡短的一行顯示過去所有的提交紀錄。
 - `git log -p`: 顯示每一次 commit 到底具體改了哪幾行程式碼 (Debug 神器)。
-

筆記優化小建議

你在 `git commit -m ""` 的部分，建議養成「語意化提交」的習慣。這會讓你的 GitHub 頁面看起來非常專業：

- `git commit -m "feat: 增加 switch 選擇結構"` (新增功能)
 - `git commit -m "fix: 修正變數 $a 漏掉分號的問題"` (修復錯誤)
 - `git commit -m "docs: 修改 Git 操作流程筆記"` (文件更新)
-

常用查詢

`git status` # 狀態
`git log` # 歷史紀錄

分支操作

`git branch <name>` # 建立分支
`git switch <name>` # 切換分支

👉 綠色星號: 目前所在分支

```
commit a2107da2c3f91346a0626c1bc8c16cb29527a39a
Author: mackliu <mackliu@gmail.com>
Date: Thu Apr 16 09:44:45 2026 +0800

    增加不同成員的獨立介紹頁面

PS D:\test> git branch
* main
PS D:\test> git branch animate
PS D:\test> git branch
    animate
    * main
PS D:\test>
```

遠端倉庫管理

git remote # 查看遠端
git remote -v # 查看詳細位置

常用操作

git remote add <名稱> <網址>
git remote set-url <名稱> <新網址>
git remote rename <舊名> <新名>
git remote remove <名稱>

Git 常用完整流程筆記(精簡但完整)

一、初始化專案(第一次建立 Repo)

建立專案資料夾後進入

cd project-folder

初始化 git(只做一次)

git init

建立說明檔(可選)

echo "# project-name" >> README.md

加入版本控制

```
git add .
```

建立第一次 commit

```
git commit -m "initial commit"
```

設定主分支名稱

```
git branch -M main
```

綁定遠端 repo

```
git remote add origin <repo網址>
```

推送到 GitHub (第一次要 -u)

```
git push -u origin main
```



二、日常開發流程(最常用)

查看狀態(必用)

```
git status
```

查看修改內容

```
git diff
```

加入檔案

```
git add <檔案>
```

git add . # ⚠ 全部加入要小心

提交版本

git commit -m "說明這次改了什麼"

推送到遠端

git push origin main



三、拉取遠端更新(避免衝突)

拉取最新版本(常用)

git pull origin main

等同於：

git fetch

git merge

👉 建議習慣：

每次開工前先 `git pull`



四、Clone 專案(下載專案)

git clone <repo網址>

cd 專案名稱



五、分支操作(基本協作)

查看分支

git branch

建立新分支

git branch dev

切換分支

git checkout dev

建立並切換(常用)

git checkout -b feature/login

合併分支

git checkout main

git merge dev

刪除分支

git branch -d dev



六、遠端設定(查看與修改)

查看遠端

git remote -v

新增遠端

git remote add origin <網址>

修改遠端網址

```
git remote set-url origin <新網址>
```

刪除遠端

```
git remote remove origin
```



七、忽略檔案(.gitignore)

建立

```
touch .gitignore
```

常見內容：

```
node_modules/
```

```
.env
```

```
.DS_Store
```



八、版本回復 / 重置(重點)

❶ 取消 **add**

```
git reset <檔案>
```

❷ 取消修改(回到上次 **commit**)

```
git checkout -- <檔案>
```

❸ 回到上一個 **commit**(保留修改)

```
git reset --soft HEAD~1
```

4 回到上一個 commit (刪除修改)

```
git reset --hard HEAD~1
```

⚠️ `--hard` 會刪掉所有變更 (危險)



九、查看紀錄

查看 commit 紀錄

```
git log
```

簡潔版

```
git log --oneline
```

查看操作紀錄 (可救回)

```
git reflog
```



十、衝突處理 (簡化流程)

當 `git pull` 或 `merge` 發生衝突:

1. 手動打開檔案修正:

```
<<<<<<< HEAD
```

```
你的版本
```

```
=====
```

```
別人的版本
```

```
>>>>>>> branch
```

2. 修正後:

```
git add .
```



```
git commit -m "fix conflict"
```

⚠️ 核心注意事項(很重要)

- `git init` 👉 一個專案只做一次
 - `git add .` 👉 會加入全部變更(要確認)
 - `git pull` 👉 開工前先做(避免衝突)
 - `commit` 訊息要清楚(方便回溯)
 - 避免過度 `commit`(但也不要完全不 `commit`)
 - `reset --hard` 👉 高風險操作
-

🧠 最小實務流程(你可以背這段)

```
git pull origin main
```

```
git add .
```

```
git commit -m "update"
```

```
git push origin main
```



不要開啟這個不然會多一個commit

📁 Git: 從雲端拉取檔案的兩種方法

方法一: 手動關聯(適合已有本地檔案或想自訂資料夾名)

1. 在目標資料夾內開啟終端機。
2. `git init` (初始化本地倉庫, 注意拼字是 `init` 而非 `inti`)。
3. `git remote add origin [網址]` (建立與雲端的連結)。
4. `git pull origin [分支名稱]` (將雲端檔案拉下來)。

方法二: 直接複製(最快、最推薦)

1. 在想要存放專案的父資料夾開啟終端機。
2. 執行 `git clone [網址]`。

- 補充：執行完後 Git 會自動建立一個與專案同名的資料夾，並完成所有初始化與遠端連結。

當你在輸入 `git commit` 時沒有加上 `-m "訊息"`，Git 就會自動跳出這個介面（通常是 **Vim** 編輯器或是你 VS Code 內建的編輯視窗），要求你輸入詳細的提交說明。

這個介面雖然看起來有點嚴肅，但它是專業開發者用來撰寫「長訊息」的地方。以下是操作步驟筆記：

Git Commit 編輯器操作流程

步驟 1：進入編輯模式

- 如果你看到像截圖中這樣的視窗，第一行通常是空白的。
- 在 **Vim** 模式下：按下鍵盤上的 **i** (代表 Insert)，左下角會出現 `-- INSERT --` 字樣，這時你才能開始打字。
- 在 **VS Code** 模式下：直接在第一行點擊並開始輸入即可。

步驟 2：撰寫 Commit 訊息

- 第一行（標題）：簡短說明這次改了什麼（建議 50 字內）。
 - 例如：`feat: 完成 PHP 選擇結構與 Switch 練習`
- 第二行：留空（這是 Git 的標準規範）。
- 第三行以後（內容）：如果你有很長的說明（例如修正了哪些 bug），可以寫在這裡。

步驟 3：儲存並離開（非常重要！）

寫完訊息後，你需要告訴 Git「我寫好了，請存檔並建立版本」：

- 如果你在 **Vim** 編輯器中：
 1. 按下 **Esc** 鍵（退出編輯模式）。
 2. 輸入 `:wq`（冒號代表指令，w 是 write 存檔，q 是 quit 離開）。
 3. 按下 **Enter**。
- 如果你在 **VS Code** 編輯標籤頁中：
 1. 直接按下 **Ctrl + S**（存檔）。
 2. 按下 **Ctrl + W** 或點擊 X 關閉該標籤頁。

筆記補充: 介面名詞解釋

根據你的截圖內容:

- **Initial commit**: 這代表這是你這個專案的第一次提交。
- **Changes to be committed**: 下方綠色的檔案(`01-basic.php` 等)就是你剛才用 `git add` 勾選進來的檔案。
- 以 `#` 開頭的行: 這些是註釋, Git 在存檔時會自動忽略它們, 不會出現在紀錄裡。

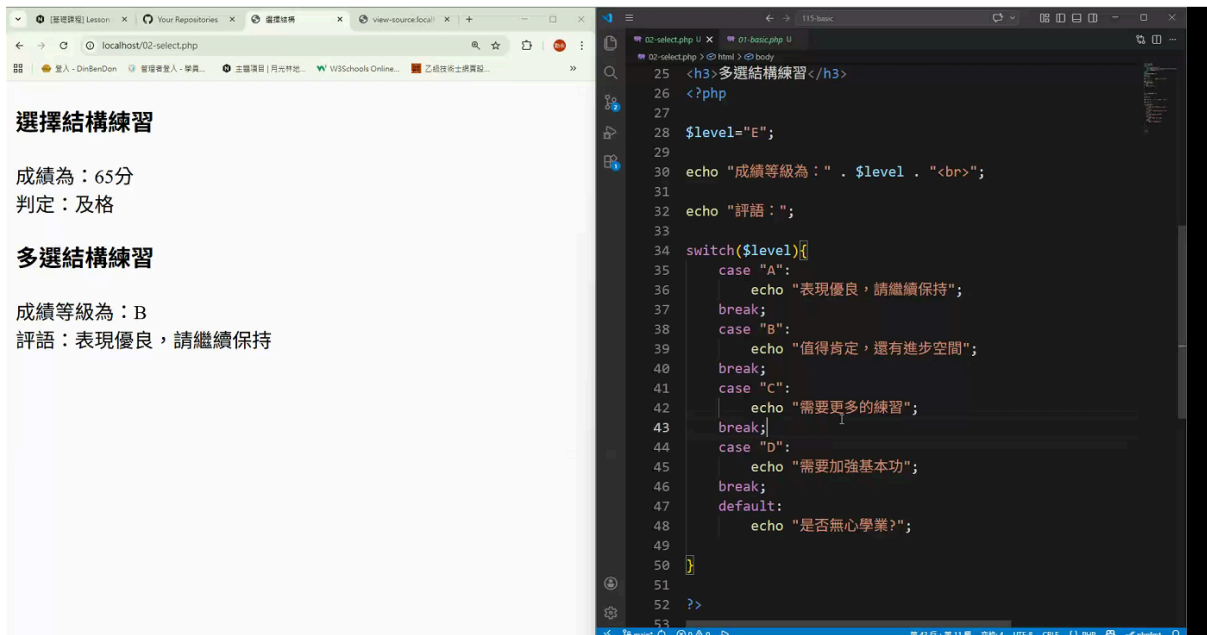
專家建議: 什麼時候用這個介面?

雖然 `git commit -m "簡短訊息"` 很方便, 但當你一次修正了非常多東西, 需要換行說明時, 就一定要用這個完整介面。

現在試試看:

1. 在第一行輸入你的訊息。
2. 按照步驟 3 儲存離開。
3. 成功後, 終端機應該會噴出 `[main (root-commit) xxx] ...` 的成功字樣!

巢狀結構最好控制在三到四層不然會太複雜



```
Date: Tue Apr 21 11:23:52 2026 +0800

new file: 01-basic.php 變數的宣告與使用及運算子、運算元
new file: 02-select.php 基本程式流程控制：條件判斷與選擇結構
PS D:\115-basic> git remote
basic
PS D:\115-basic> git branch
* main
PS D:\115-basic> git push basic main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 24 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 978 bytes | 978.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mackliu/11501-FULL-BASIC.git
* [new branch] main -> main
```

Git 遠端存取與同步筆記

一、下載遠端儲存庫 (Clone)

當你「第一次」要將雲端專案下載到電腦時，使用 `git clone`。

- 標準下載方式：
- Bash

`git clone https://github.com/chengkk0910/web2026-04-22.git`

- 預設會建立一個與專案名稱相同的資料夾 (`web2026-04-22`)。
- 自定義資料夾名稱：
如果你想在下載時順便更改資料夾名稱，在網址後方加上名稱即可：
- Bash

```
# 下載後資料夾名稱會變成 kai55688
git clone https://github.com/chengkk0910/web2026-04-22.git kai55688
```

```
# 下載後資料夾名稱會變成 kai12345
git clone https://github.com/chengkk0910/web2026-04-22.git kai12345
```

-
-

二、同步最新進度 (Pull)

當你「第二次以後」要獲取同學或老師更新的檔案時，不需重複 **Clone**，只要進入該資料夾執行：

```
Bash
git pull
```

原理：這會比對雲端與你電腦的差異，只下載新增或修改的部分，效率比重新 Clone 快得多。

三、操作介面小撇步

在 Windows 系統中，最直覺的開啟方式：

1. 進入你想放專案的資料夾。
2. 在空白處 按右鍵。
3. 點擊「**Git Bash Here**」。
 - 這會直接在當前路徑打開終端機，省去輸入 `cd` 切換路徑的麻煩。

核心指令差異表

指令	使用時機	效果
<code>git clone [url]</code>	第一次下載專案	下載完整專案並建立資料夾
<code>git pull</code>	第二次以後更新	下載遠端的新版本並與本地合併

專業建議 (Pro Tips)

既然你下載了老師的 `web2026-04-22.git`，通常裡面的內容會隨課程更新：

1. 先確認路徑：執行 `git pull` 前，一定要先確認你在專案資料夾內（Bash 視窗藍色的文字結尾應該是該專案名）。

2. 避免衝突: 如果你自己有修改老師的檔案, `git pull` 可能會提示衝突 (Conflict)。如果你只是要單純看老師的最新程式碼, 建議先將自己的檔案 `commit` 存檔後再 `pull`。



按右鍵=>打開 git Bash

```
User@1113-Teacher MINGW64 /d
$ git clone https://github.com/chengkk0910/web2026-04-22.git kai55688
Cloning into 'kai55688'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

User@1113-Teacher MINGW64 /d
$ git clone https://github.com/chengkk0910/web2026-04-22.git kai12345
Cloning into 'kai12345'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

User@1113-Teacher MINGW64 /d
$
```

```
User@1113-Teacher MINGW64 /d
```

```
$ ls
```

```
'$RECYCLE.BIN' /
```

```
115-basic/
```

```
User@1113-Teacher MINGW64 /d
```

```
$ cd web2026-04-22/
```

```
User@1113-Teacher MINGW64 /d/web2026-04-22 (main)
```

```
$ |
```

```
269 git clone https://github.com/chengkk0910/web2026-04-22.git kai55688
270 git clone https://github.com/chengkk0910/web2026-04-22.git kai12345
271 cls
272 git clone https://github.com/chengkk0910/web2026-04-22.git
273 ls
274 cd web2026-04-22/
275 cd ..
276 ls
277 git clone https://github.com/chengkk0910/web2026-04-22.git
278 cd web2026-04-22/
279 ls
280 history
```

```
User@1113-Teacher MINGW64 /d/web2026-04-22 (main)
```

```
● $ git add .
```

```
User@1113-Teacher MINGW64 /d/web2026-04-22 (main)
```

```
● $ git commit -m"init 0422"
```

```
[main c48c813] init 0422
```

```
13 files changed, 214 insertions(+)
```

```
create mode 100644 s20260414/apple.docx
```